



**SWAMINARAYAN
UNIVERSITY**
THE RESERVOIR OF ALL KNOWLEDGE

**Faculty of Engineering (Degree)
Semester – 7th**

Subject Name – Information security

Subject Code – BE103707

Faculty Name – Radhika Gokani

CHAPTER -2

1. Stream ciphers and block ciphers

These are two fundamental types of symmetric encryption used to protect data. In **information security**, stream and block ciphers are useful tools for **symmetric encryption**, meaning the same key is used for both encryption and decryption.

Stream Ciphers

- **Encrypt data one bit or byte at a time**, making them ideal for continuous or real-time data transmission
- Use a **keystream**, generated from a secret key, which is XORed with the plaintext to produce ciphertext
- Often used in **low-latency applications** like voice calls, video streaming, or secure messaging
- Examples: RC4 (legacy), ChaCha20, Salsa20

Security Note: Stream ciphers are fast and efficient, but if the same keystream is reused, it can lead to vulnerabilities.

Block Ciphers

- **Encrypt fixed-size blocks** of data (e.g., 64 or 128 bits) using complex mathematical transformations
- Require **modes of operation** (like ECB, CBC, CTR) to handle data larger than a single block
- Commonly used for **file encryption, secure storage, and HTTPS**
- Examples: AES, DES (deprecated), Blowfish

Security Note: Block ciphers offer strong security when used with proper modes and padding. They are more resistant to certain attacks when the same key is reused.

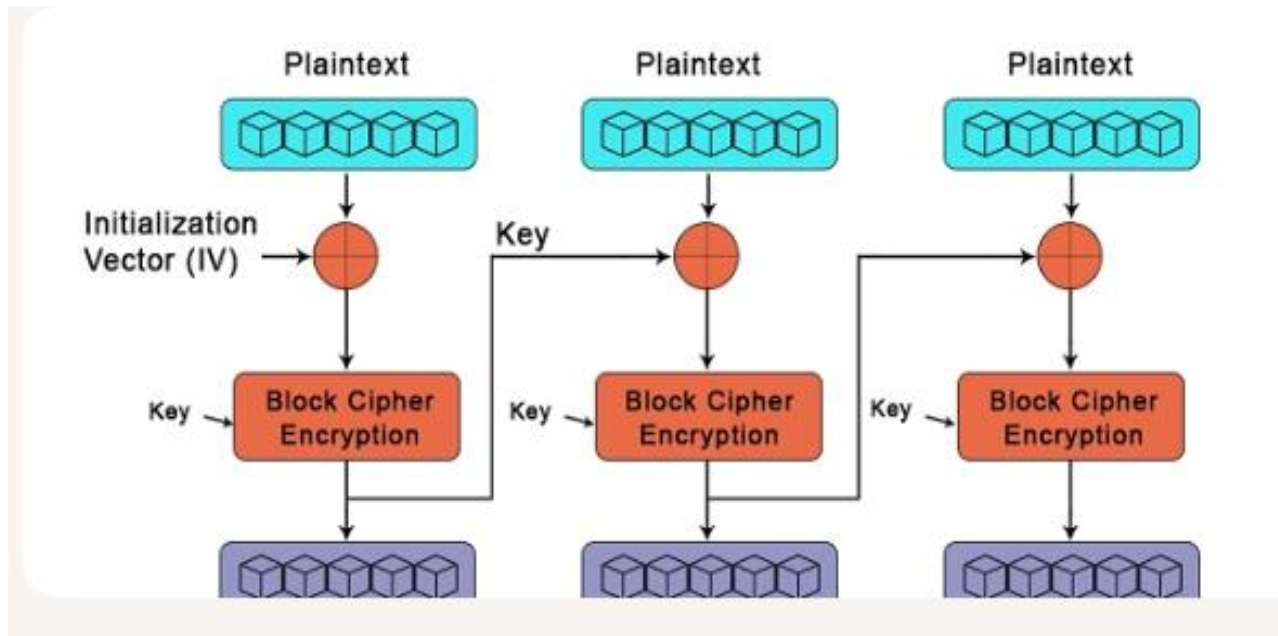
Comparison Table

Feature	Stream Cipher	Block Cipher
Data Handling	Bit/byte at a time	Fixed-size blocks
Speed	Faster	Slower
Use Case	Real-time data (e.g., calls)	Bulk data (e.g., files, HTTPS)
Key Reuse Risk	High (if keystream reused)	Lower (with proper mode)
Complexity	Simpler	More complex
Examples	RC4, ChaCha20	AES, DES, Blowfish

2. Block Cipher structure

Security Features

- **Avalanche Effect:** A small change in input causes a large change in output.
- **Non-linearity:** S-boxes must be non-linear to resist linear attacks.
- **Large Key and Block Sizes:** Typically 128-bit or more to prevent brute-force attacks.



Block Cipher Structure Explained

A block cipher is a symmetric key encryption algorithm that transforms fixed-size blocks of plaintext into ciphertext using a cryptographic key.

Basic Components

- **Plaintext Block:** Input data divided into fixed-size blocks (e.g., 64 or 128 bits).
- **Key:** A secret key used for both encryption and decryption.
- **Encryption Function (F):** A complex function applied to the data and key.
- **Rounds:** Multiple iterations of the encryption function to increase security.
- **Ciphertext Block:** The encrypted output of the same size as the plaintext block.

General Workflow

1. **Initial Input:** Plaintext block and key are fed into the cipher.
2. **Round Processing:**
 - Each round applies the function F to part of the data and the key.
 - The output is mixed with the other half of the data (often using XOR).
 - The halves are swapped or combined depending on the cipher design.
3. **Final Output:** After all rounds, the result is the ciphertext block.

Common Structures

Structure Type	Description
Feistel Network	Splits data into two halves; only one half is processed per round.
Substitution-Permutation Network (SPN)	Applies substitution (S-boxes) and permutation (P-boxes) in each round.

Example: Feistel Cipher Round

plaintext

$L_0, R_0 = \text{split}(\text{plaintext})$

for i in range(n_{rounds}):

$L_i = R_{i-1}$

$R_i = L_{i-1} \text{ XOR } F(R_{i-1}, K_i)$

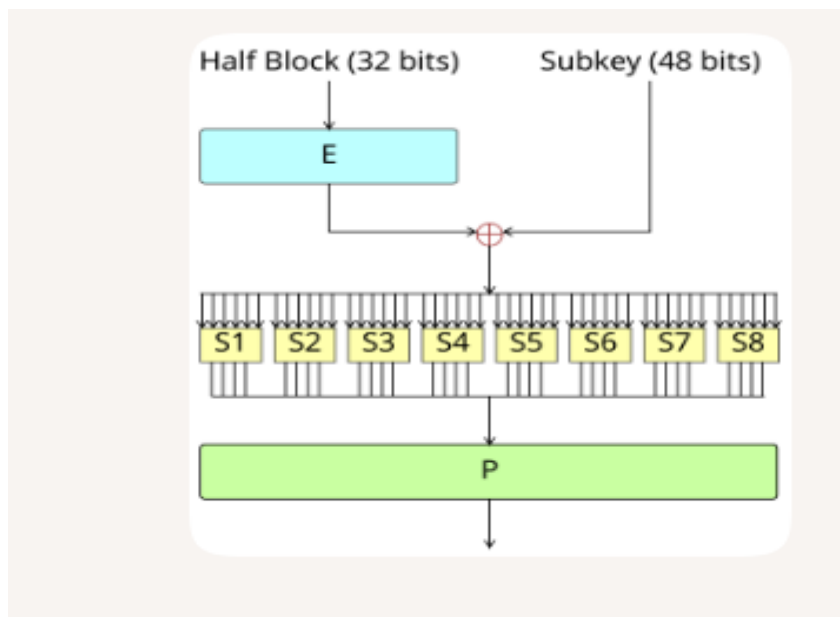
Where K_i is the round key derived from the main key.

3. Data Encryption standard (DES) with example

DES is a symmetric-key block cipher developed in the 1970s to secure sensitive data. It operates on 64-bit blocks of plaintext and uses a 56-bit key (plus 8 parity bits) to produce 64-bit ciphertext.

Features

- **Block Size:** 64 bits
- **Key Size:** 56 bits (effective)
- **Rounds:** 16 Feistel rounds
- **Structure:** Based on Feistel network (splits data into two halves)



Encryption Process

1. **Initial Permutation (IP):** Rearranges the 64-bit plaintext using a fixed table.
2. **Feistel Rounds (16 total):**
 - Split into Left (L) and Right (R) halves.
 - Apply round function $F(R, K)$ to the right half and XOR with the left.
 - Swap halves for the next round.
3. **Final Permutation (IP^{-1}):** Inverse of the initial permutation to produce ciphertext.

Step-by-Step Breakdown:

Step	Description
Plaintext (Hex)	0123456789ABCDEF

Step	Description
Plaintext (Binary)	00000001 00100011 01000101 01100111 10001001 10101111 11001101 11101111
Key (Hex)	133457799BBCDFF1
Key (Binary)	Converted to 56-bit by removing parity bits
Initial Permutation	Bits rearranged using IP table
Rounds	16 rounds of Feistel structure using subkeys
Final Permutation	Produces final ciphertext

Resulting Ciphertext (Hex): 85E813540F0AB405

This output is the encrypted version of the original plaintext using DES.

Security Note

DES is now considered insecure due to its short key length and vulnerability to brute-force attacks. It has been replaced by **AES** and **Triple DES (3DES)** in modern systems.

4. Strength of DES

Despite being considered outdated today, DES had several notable strengths when it was introduced in the 1970s. Although DES is no longer considered secure by modern standards, it was a ground breaking algorithm in its time.

Technical Strengths

- **Feistel Network Structure:** DES uses a robust Feistel structure with 16 rounds of encryption, providing strong diffusion and confusion—key principles in cryptography.
- **Simplicity and Speed:** Its design allowed for fast encryption and decryption, especially in hardware implementations, making it ideal for early computing environments.
- **Fixed Block Size:** Operates on 64-bit blocks, which was sufficient for many applications at the time.

Algorithmic Design Strengths

- **Feistel Network Architecture:** DES uses a 16-round Feistel structure, which ensures strong diffusion and confusion—essential for secure encryption.
- **Sub key Generation:** From a 56-bit key, DES derives 16 unique 48-bit subkeys, adding complexity to each round and making cryptanalysis harder.
- **S-Boxes:** The substitution boxes (S-boxes) used in DES were carefully designed to resist differential cryptanalysis, even before the technique was publicly known².

Performance and Efficiency

- **Fast Execution:** DES was optimized for speed, especially in hardware, making it suitable for real-time applications like ATMs and secure communications.
- **Low Resource Requirements:** Its simplicity allowed implementation on limited hardware, which was crucial in the 1970s and 1980s.

Security Features

- **Sub key Generation:** DES derives 16 different 48-bit sub keys from the main 56-bit key, adding complexity to each round.
- **Resistance to Known Attacks (Initially):** When first released, DES was resistant to many cryptanalytic attacks known at the time, including brute-force attacks—though this changed as computing power increased².

5. Design principles of block cipher

Block ciphers are built to securely transform fixed-size blocks of plaintext into ciphertext using a symmetric key. Their design is guided by several foundational principles to ensure **confidentiality**, **resistance to attacks**, and **efficient performance**.

principles:

1. Confusion and Diffusion

- **Confusion:** Obscures the relationship between the key and ciphertext. Achieved through substitution (e.g., S-boxes).
- **Diffusion:** Spreads the influence of each plaintext bit across the ciphertext. Achieved via permutations and mixing operations.
- These principles were introduced by Claude Shannon and are fundamental to resisting statistical attacks.

2. Number of Rounds

- Multiple rounds of encryption increase complexity and security.
- Example: DES uses 16 rounds; AES uses 10, 12, or 14 depending on key size.
- More rounds enhance resistance to brute-force and differential attacks.

3. Round Function (Function F)

- Should be **nonlinear**, **complex**, and exhibit:
 - **Avalanche Effect:** A small change in input causes a large change in output.
 - **Bit Independence:** Output bits change independently with input changes.
- A strong round function makes cryptanalysis significantly harder².

4. Key Schedule Algorithm

- Generates round keys from the main key.
- Must ensure:
 - **Unpredictability** and **independence** of round keys.
 - Resistance to weak keys and related-key attacks.
- Example: DES uses a 56-bit key split into two 28-bit halves for scheduling.

5. Block Size

- Larger block sizes (e.g., 128 bits) reduce vulnerability to statistical and brute-force attacks.
- Common sizes: 64 bits (DES), 128 bits (AES).
- Must balance security with performance².

6. S-Box Design

- S-boxes introduce nonlinearity and confusion.
- Should be:
 - Resistant to **linear** and **differential cryptanalysis**.
 - Designed to maximize avalanche effect.
- Poorly designed S-boxes can compromise the entire cipher.

7. Security Analysis

- Ciphers must be tested against:
 - **Differential cryptanalysis**
 - **Linear cryptanalysis**
 - **Brute-force attacks**
 - **Side-channel attacks**
- Formal proofs and empirical testing are essential before deployment².

8. Efficiency and Simplicity

- Should be fast in both hardware and software.
- Simpler designs are easier to implement and verify.
- Trade-offs between speed and security must be carefully managed.

9. Flexibility and Mode Compatibility

- Must support various modes of operation:
 - ECB, CBC, CFB, OFB, CTR
- Flexibility allows adaptation to different applications and data types.

6. AES with structure

AES is a symmetric block cipher that encrypts data in **128-bit blocks** using keys of **128, 192, or 256 bits**. It's based on a **substitution-permutation network**, not a Feistel structure, meaning it processes the entire data block as a matrix rather than splitting it.

Basic Structure Overview

- **Block Size:** 128 bits (16 bytes)
- **Key Sizes:** 128, 192, or 256 bits
- **Rounds:**
 - 10 for 128-bit key
 - 12 for 192-bit key
 - 14 for 256-bit key
- **Data Representation:** 4×4 matrix of bytes called the **State**

Encryption Process Steps

Each round (except the final one) includes four operations:

Step	Description
SubBytes	Byte-by-byte substitution using a nonlinear S-box
ShiftRows	Cyclically shifts rows of the State matrix to the left
MixColumns	Mixes each column using matrix multiplication over GF(2 ⁸)
AddRoundKey	XORs the State with a round key derived from the main key

- **Initial Round:** Only AddRoundKey
- **Final Round:** SubBytes, ShiftRows, AddRoundKey (MixColumns skipped)

Decryption Process

- Inverse operations:
 - **InvSubBytes**
 - **InvShiftRows**
 - **InvMixColumns**
 - **AddRoundKey** (same as encryption due to XOR symmetry)
- Round keys are applied in reverse order

AES is widely used in VPNs, Wi-Fi security, file encryption, and secure communications due to its speed and robustness.

7. Its transformation functions, key expansion, example and implementation

AES (Advanced Encryption Standard) uses a series of transformation functions in each round of encryption and decryption to ensure **confusion**, **diffusion**, and **security**. These transformations operate on a 4×4 byte matrix called the **State**.

1. SubBytes (Byte Substitution)

- **Purpose:** Introduces nonlinearity and confusion.
- **How it works:**
 - Each byte in the State is replaced using a fixed **S-box** (substitution box).
 - The S-box is designed to resist linear and differential cryptanalysis.
- **Inverse:** **InvSubBytes** uses an inverse S-box during decryption.

2. ShiftRows (Row Shifting)

- **Purpose:** Provides diffusion by rearranging bytes.
- **How it works:**
 - Row 0: No shift
 - Row 1: Shift left by 1 byte
 - Row 2: Shift left by 2 bytes
 - Row 3: Shift left by 3 bytes
- **Inverse:** **InvShiftRows** shifts rows to the right during decryption.

3. MixColumns (Column Mixing)

- **Purpose:** Enhances diffusion across columns.
- **How it works:**
 - Each column is treated as a polynomial and multiplied by a fixed matrix over $GF(2^8)$.
 - This mixes the bytes within each column.
- **Inverse:** **InvMixColumns** uses a different matrix for decryption.

4. AddRoundKey (Key Mixing)

- **Purpose:** Integrates the encryption key into the data.
- **How it works:**
 - Each byte of the State is XORed with the corresponding byte of the round key.
 - Performed before the first round and in every round.
- **Inverse:** Same operation used in decryption due to XOR symmetry.

These four transformations are repeated across multiple rounds (10, 12, or 14 depending on key size). Together, they ensure that AES is secure, efficient, and resistant to known attacks.

Key Expansion (Key Schedule)

- Generates **round keys** from the original key using Rijndael's algorithm
- Involves:
 - **RotWord:** rotates bytes in a word
 - **SubWord:** applies S-box substitution
 - **Rcon:** round constants added for nonlinearity
- Produces **N+1 round keys**, each 128 bits

Example Setup

simple AES encryption example **using a 128-bit key and plaintext block. AES operates on a 4×4 byte matrix called the State, and applies a series of transformations over multiple rounds.**

- **Key (128-bit):** "Thats my Kung Fu" → Hex: 54 68 61 74 73 20 6D 79 20 4B 75 6E 67 20 46 75
- **Plaintext (128-bit):** "Two One Nine Two" → Hex: 54 77 6F 20 4F 6E 65 20 4E 69 6E 65 20 54 77 6F

These are both 16 ASCII characters, each 1 byte, forming 128-bit blocks.
